



## 物理结构设计&系统实现

课题名称: 酒店管理系统

课题组员: 周旻旻 程什开 陈婧琳 陈雨杰

完成时间: 5.23

分工:

本章按照系统功能模块分工:

|            |            |
|------------|------------|
| 客房资源管理-周旻旻 | 支付财务管理-程什开 |
| 后台管理-陈婧琳   | 预定入住管理-陈雨杰 |

排版整合: 周旻旻

## 6 物理设计

本系统以 MySQL 数据库作为物理实现平台，数据库名称为 `hotel_management`，字符集统一采用 `utf8mb4`，表存储引擎采用 `InnoDB`。该设计能够支持事务处理、行级锁、外键约束和中文数据存储，适合酒店预订、入住、退房结算、评价统计等连续业务场景。

物理结构设计按成员负责的功能模块划分：客房资源与房态管理、预订与入住管理、账单支付与财务统计、后台管理评价与统计。各模块通过主键、唯一约束、外键、触发器和操作日志共同维护数据一致性。

### 6.1 物理存储与索引原则

主键字段统一使用 `bigint` 自增形式，业务编号类字段使用唯一索引保证可追溯性；高频筛选字段如房态、订单状态、支付状态、员工编号、客户编号和时间字段建立普通索引；多表关联频繁的字段建立外键索引，以减少视图查询、存储过程调用和统计报表的扫描成本。

| 设计项  | 实现方式                                    | 说明                                                                                                                             |
|------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 字符集  | <code>utf8mb4/utf8mb4_general_ci</code> | 支持中文、图片路径、评价内容等多语言文本。                                                                                                          |
| 存储引擎 | <code>InnoDB</code>                     | 支持事务、外键、行级锁和崩溃恢复。                                                                                                              |
| 主键   | <code>bigint auto_increment</code>      | 各实体使用数字主键，便于关联与扩展。                                                                                                             |
| 唯一约束 | 业务编号、账号、流水号                             | 防止订单编号、支付流水、退款流水、登录账号重复。                                                                                                       |
| 状态字段 | <code>varchar(30)</code>                | 便于表达 <code>available</code> 、 <code>reserved</code> 、 <code>occupied</code> 、 <code>paid</code> 、 <code>refunded</code> 等业务状态。 |
| 时间字段 | <code>timestamp/datetime</code>         | 记录创建、更新、入住、退房、支付、清扫和维修节点。                                                                                                      |

表 1 物理设计总体原则

## 6.2 客房资源与房态管理索引设计

客房资源模块的索引重点是“查可订房、查当前房态、查房态变更历史”。因此索引围绕客房目录、实体房间、价格策略、房态日志、清扫任务和维修记录展开。

| 表名                 | 索引名                                                | 索引列                                         | 设计说明               |
|--------------------|----------------------------------------------------|---------------------------------------------|--------------------|
| room_catalog       | idx_room_catalog_category / idx_room_catalog_hotel | room_category_id / hotel_profile_id         | 提升房型、酒店维度筛选和关联查询。  |
| room_instance      | uk_room_instance_no / idx_room_instance_status     | room_no / status                            | 保证房号唯一，并加快可预订房间检索。 |
| price_policy       | idx_price_policy_catalog_date                      | room_catalog_id,start_date,end_date,enabled | 支持按日期匹配可用价格策略。     |
| room_status_log    | idx_room_status_room_time                          | room_id,created_at                          | 便于查询某房间状态变化轨迹。     |
| housekeeping_task  | idx_housekeeping_room_status                       | room_id,task_status                         | 快速定位待清扫或正在清扫任务。    |
| maintenance_record | idx_maintenance_room_status                        | room_id,maintenance_status                  | 支撑房间维修状态跟踪。        |

表 2 客房资源与房态管理主要索引

## 6.3 预订与入住管理索引设计

预订与入住模块的查询压力主要集中在客户订单检索、入住日期范围判断、订单到房间的分配关系以及入住记录追踪。SQL 在保留原有预订订单表的基础上，补充了 booking\_room、booking\_guest、checkin\_record 和 checkin\_guest 等规范化业务表，因此索引设计同时服务于前端订单管理、后端入住办理和数据库存储过程调用。

| 表名            | 索引名                                        | 索引列                         | 设计说明                 |
|---------------|--------------------------------------------|-----------------------------|----------------------|
| booking_order | yudingbianhao / idx_booking_order_customer | yudingbianhao / customer_id | 保证订单编号唯一，支持客户维度订单查询。 |

| 表名             | 索引名                                                    | 索引列                                                                   | 设计说明                |
|----------------|--------------------------------------------------------|-----------------------------------------------------------------------|---------------------|
| booking_order  | idx_booking_order_catalog /<br>idx_booking_order_dates | room_catalog_id /<br>expected_checkin_date,<br>expected_checkout_date | 支持按客房目录和预计入住日期检索订单。 |
| booking_room   | uk_booking_room                                        | booking_id, room_id                                                   | 避免同一订单重复关联同一实体房间。   |
| booking_room   | idx_booking_room_room_date                             | room_id, stay_start_date,<br>stay_end_date                            | 支撑可订房校验和房间日期冲突判断。   |
| booking_guest  | idx_booking_guest_booking                              | booking_id                                                            | 快速读取订单下的入住人预登记信息。   |
| checkin_record | idx_checkin_room /<br>idx_checkin_customer             | room_id / customer_id                                                 | 支持按房间或客户追踪入住记录。     |
| checkin_guest  | fk_checkin_guest_record                                | checkin_id                                                            | 通过外键索引读取入住人明细。      |

表 3 预订与入住管理主要索引

## 6.4 账单支付与财务统计索引设计

账单支付模块关注退房账单生成、支付流水登记、退款回写和经营收入统计。bill 表保存结算汇总，payment\_record 与 refund\_record 保存明细流水；触发器负责在支付和退款成功后自动回写账单金额、账单状态以及预订支付状态，因此相关索引优先覆盖账单编号、客户编号、支付流水号和退款流水号。

| 表名             | 索引名                                     | 索引列                         | 设计说明                 |
|----------------|-----------------------------------------|-----------------------------|----------------------|
| bill           | idx_bill_checkin /<br>idx_bill_customer | checkin_id /<br>customer_id | 按入住记录生成账单，并支持客户消费统计。 |
| payment_record | uk_payment_no                           | payment_no                  | 保证支付流水号唯一，便于财务核对。    |
| payment_record | idx_payment_bill                        | bill_id                     | 快速汇总某账单下的支付记录。       |
| refund_record  | uk_refund_no                            | refund_no                   | 保证退款流水号唯一。           |
| refund_record  | idx_refund_payment                      | payment_id                  | 退款与原支付记录关联，支持原路追溯。   |

| 表名            | 索引名                          | 索引列   | 设计说明                                    |
|---------------|------------------------------|-------|-----------------------------------------|
| booking_order | chk_booking_order_pay_status | ispay | 约束支付状态只能取 unpaid、partial、paid、refunded。 |

表 4 账单支付与财务统计主要索引

## 6.5 后台管理、评价与统计索引设计

后台管理模块承担基础数据维护、评价反馈、收藏、操作审计和员工绩效分析。SQL 对评价分数、收藏唯一性、令牌唯一性、系统参数唯一性和员工绩效周期都增加了约束，同时评价触发器已经补充 operation\_log 写入逻辑，便于把用户反馈行为纳入后续审计和统计。

| 表名                   | 索引名                                            | 索引列                                         | 设计说明                        |
|----------------------|------------------------------------------------|---------------------------------------------|-----------------------------|
| employee             | employeeegonghao                               | employeeegonghao                            | 保证员工工号唯一。                   |
| favorite             | uk_favorite_user_ref/<br>idx_favorite_user     | userid, tablename,<br>refid / userid        | 避免同一客户重复收藏同一对象，并提升收藏列表查询效率。 |
| room_review          | idx_room_review_ref/<br>idx_room_review_user   | refid / userid                              | 支持按客房或客户查询评价。               |
| hotel_review         | idx_hotel_review_ref/<br>idx_hotel_review_user | refid / userid                              | 支持按酒店或客户查询评价。               |
| auth_token           | uk_auth_token_value                            | auth_token                                  | 保证登录令牌唯一，配合过期时间约束。          |
| system_config        | uk_system_config_name                          | name                                        | 保证系统配置项名称唯一。                |
| operation_log        | idx_operation_operator                         | operator_type,<br>operator_id               | 按操作者统计关键业务行为。               |
| employee_performance | uk_employee_performance_period                 | employee_id,<br>period_start,<br>period_end | 避免同一员工同一考核周期重复生成绩效记录。       |

表 5 后台管理、评价与统计主要索引

## 7 系统 SQL 实现

### 7.1 完整性设计

本系统包含 29 张数据表、10 个视图、11 个存储过程、24 个触发器。完整性设计通过主键、唯一约束、外键、检查约束、存储过程事务和触发器联动共同完成。

**实体完整性：**所有核心业务表均以 id 或自增 bigint 字段作为主键；订单编号、房间号、支付流水号、退款流水号、客户账号、员工工号、登录令牌等具有业务唯一性的字段使用唯一约束控制重复数据。

**参照完整性：**规范化扩展表通过外键连接原有业务表，例如 room\_instance 关联 room\_catalog, booking\_order 关联 customer、room\_catalog 和 room\_instance, bill 关联 booking\_order、checkin\_record 与 customer。

**用户自定义完整性：**脚本含有大量 CHECK 约束，对房态、订单状态、支付状态、金额非负、日期先后、评分范围、员工绩效等级等规则进行数据库层校验，避免只依赖前端校验。

**过程与触发器完整性：**存储过程统一封装可订房检索、预订创建、入住确认、账单生成和支付登记；触发器自动维护房态、账单金额、预订支付状态、评分汇总和操作日志，减少跨表状态不一致。

| 完整性类别 | 涉及对象                                                       | 实现方式        | 说明                         |
|-------|------------------------------------------------------------|-------------|----------------------------|
| 主键完整性 | 29 张业务表                                                    | PRIMARY KEY | 每张表均设置主键，用于唯一标识业务记录。       |
| 唯一约束  | booking_order、room_instance、payment_record、refund_record 等 | UNIQUE KEY  | 控制订单号、房号、流水号等业务编号不重复。      |
| 外键约束  | 预订、入住、账单、评价、清扫、维修、绩效表                                      | FOREIGN KEY | 保证子表记录必须依赖有效主表记录。          |
| 取值约束  | 状态、评分、金额、日期字段                                              | CHECK       | 限制状态枚举、评分 1 到 5、金额非负和日期合法。 |
| 业务联动  | 房态、账单、评分、日志                                                | TRIGGER     | 在插入或更新时自动回写汇总状态和日志。        |
| 事务封装  | 预订、入住、支付等过程                                                | PROCEDURE   | 将多表写入放入统一业务过程，降低半完成风险。     |

表 6 主要完整性约束说明

```
ALTER TABLE `booking_order`
  ADD CONSTRAINT `chk_booking_order_pay_status` CHECK (`ispay` IN
('unpaid','partial','paid','refunded')),
  ADD CONSTRAINT `chk_booking_order_status` CHECK (`booking_status` IN
('created','checked_in','cancelled','completed'));

ALTER TABLE `bill`
  ADD CONSTRAINT `chk_bill_refund_not_exceed_paid` CHECK (`refund_amount` <=
`paid_amount`),
  ADD CONSTRAINT `chk_bill_status` CHECK (`status` IN
('unpaid','partial','paid','refunded'));
```

## 7.2 表的创建

建表脚本以 `hotel_management_answer.sql` 为准。主要业务内容运营表为用于处理实体房间、价格策略、订单房间明细、入住记录、账单支付、清扫维修和员工绩效，并有酒店简介、客房、客户、员工、公告、评价、收藏、令牌等基本信息表。已经搭配兼容了前端页面，也能支撑后续后端添加的 `/ops` 数据库操作接口。

| 模块   | 表名            | 主键/主要约束                        | 用途说明                         |
|------|---------------|--------------------------------|------------------------------|
| 基础配置 | system_config | PK: id; uk_system_config_name  | 保存轮播图、图片路径等系统参数。             |
| 基础配置 | hotel_profile | PK: id                         | 保存酒店名称、星级、地址、服务热线和评分汇总。      |
| 基础配置 | notice        | PK: id                         | 保存酒店公告和运营通知。                 |
| 用户权限 | admin_user    | PK: id; role 默认 admin          | 保存管理员账号，不再误写为 username 唯一约束。 |
| 用户权限 | customer      | PK: id; UNIQUE customerming    | 保存客户账号、姓名和联系方式。              |
| 用户权限 | employee      | PK: id; UNIQUE employeeconghao | 保存员工档案、岗位和入职信息。              |
| 用户权限 | auth_token    | PK: id; uk_auth_token_value    | 保存登录令牌及过期时间。                 |
| 客房资源 | room_category | PK: id; UNIQUE room_category   | 维护客房类型字典。                    |
| 客房资源 | room_catalog  | PK: id; 外键到酒店和房型               | 保存面向销售展示的客房目录。               |

| 模块   | 表名                 | 主键/主要约束                                   | 用途说明              |
|------|--------------------|-------------------------------------------|-------------------|
| 客房资源 | room_instance      | PK: id; UNIQUE room_no                    | 保存可实际分配入住的实体房间。   |
| 客房资源 | price_policy       | PK: id; FK room_catalog_id                | 保存节假日价、周末价等价格策略。  |
| 客房资源 | room_status_log    | PK: id; FK room_id                        | 记录房态变更轨迹。         |
| 预订入住 | booking_order      | PK: id; UNIQUE yudingbianhao              | 保存客户预订订单和支付/订单状态。 |
| 预订入住 | booking_room       | PK: id; uk_booking_room                   | 保存订单与实体房间的分配关系。   |
| 预订入住 | booking_guest      | PK: id; FK booking_id                     | 保存预订阶段的入住人预登记。    |
| 预订入住 | booking_status_log | PK: id; FK booking_id                     | 记录订单状态变化。         |
| 预订入住 | checkin_assignment | PK: id                                    | 保留原入住安排表，兼容已有页面。  |
| 预订入住 | checkin_record     | PK: id; FK booking/room/customer/employee | 保存正式入住主记录。        |
| 预订入住 | checkin_guest      | PK: id; FK checkin_id                     | 保存入住人明细。          |
| 支付财务 | bill               | PK: id; FK booking/checkin/customer       | 保存退房结算账单和金额汇总。    |
| 支付财务 | payment_record     | PK: id; UNIQUE payment_no                 | 保存支付流水。           |
| 支付财务 | refund_record      | PK: id; UNIQUE refund_no                  | 保存退款流水。           |
| 后台服务 | favorite           | PK: id; uk_favorite_user_ref              | 保存客户收藏酒店或客房的记录。   |
| 后台服务 | room_review        | PK: id; 评分 CHECK                          | 保存客房评价和评分。        |
| 后台服务 | hotel_review       | PK: id; 评分 CHECK                          | 保存酒店评价和评分。        |
| 后台服务 | operation_log      | PK: id; idx_operation_operator            | 保存关键业务操作日志。       |
| 后台服务 | housekeeping_task  | PK: id; FK room/checkin/employee          | 保存清扫任务。           |
| 后台服务 | maintenance_record | PK: id; FK room/employee                  | 保存维修记录。           |



| 模块   | 表名                   | 主键/主要约束                                   | 用途说明        |
|------|----------------------|-------------------------------------------|-------------|
| 后台服务 | employee_performance | PK: id;<br>uk_employee_performance_period | 保存员工周期绩效考核。 |

表 7 数据表创建与功能模块对应

```

CREATE TABLE `room_instance` (
  `id` bigint NOT NULL AUTO_INCREMENT,
  `room_catalog_id` bigint NOT NULL, `room_no` varchar(50) NOT NULL,
  `floor_no` int NOT NULL, `status` varchar(30) NOT NULL DEFAULT 'available',
  PRIMARY KEY (`id`), UNIQUE KEY `uk_room_instance_no` (`room_no`),
  KEY `idx_room_instance_catalog` (`room_catalog_id`), KEY `idx_room_instance_status`
  (`status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `booking_room` (
  `id` bigint NOT NULL AUTO_INCREMENT, `booking_id` bigint NOT NULL,
  `room_id` bigint NOT NULL, `room_price` decimal(10,2) NOT NULL,
  `stay_start_date` date NOT NULL, `stay_end_date` date NOT NULL,
  `room_status` varchar(30) NOT NULL DEFAULT 'reserved',
  PRIMARY KEY (`id`), UNIQUE KEY `uk_booking_room` (`booking_id`,`room_id`),
  KEY `idx_booking_room_room_date` (`room_id`,`stay_start_date`,`stay_end_date`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `bill` (
  `id` bigint NOT NULL AUTO_INCREMENT, `booking_id` bigint DEFAULT NULL,
  `checkin_id` bigint DEFAULT NULL, `customer_id` bigint DEFAULT NULL,
  `room_amount` decimal(10,2) NOT NULL DEFAULT 0.00,
  `payable_amount` decimal(10,2) NOT NULL DEFAULT 0.00,
  `paid_amount` decimal(10,2) NOT NULL DEFAULT 0.00,
  `refund_amount` decimal(10,2) NOT NULL DEFAULT 0.00, `status` varchar(30) NOT NULL DEFAULT
  'unpaid',
  PRIMARY KEY (`id`), KEY `idx_bill_checkin` (`checkin_id`), KEY `idx_bill_customer`
  (`customer_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `payment_record` (
  `id` bigint NOT NULL AUTO_INCREMENT, `bill_id` bigint NOT NULL,
  `payment_no` varchar(80) NOT NULL, `pay_method` varchar(30) NOT NULL,
  `pay_amount` decimal(10,2) NOT NULL, `pay_status` varchar(30) NOT NULL DEFAULT 'success',
  PRIMARY KEY (`id`), UNIQUE KEY `uk_payment_no` (`payment_no`), KEY `idx_payment_bill`
  (`bill_id`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

## 7.3 安全性设计

数据库安全性采用分用户授权方式实现。脚本创建 `hotel_admin`、`hotel_staff`、`hotel_report` 和 `hotel_customer_app` 四类数据库用户，使后台维护、前台业务处理、报表查询和客户侧访问分离。这种设计与前端管理中心、业务过程工具和报表页面相对应，避免普通用户直接拥有全库权限。

| 数据库用户                           | 角色定位   | 主要权限                                         | 权限说明                        |
|---------------------------------|--------|----------------------------------------------|-----------------------------|
| <code>hotel_admin</code>        | 数据库管理员 | ALL PRIVILEGES                               | 用于数据库初始化、维护和课程演示时的全库管理。     |
| <code>hotel_staff</code>        | 酒店员工   | 业务表<br>SELECT/INSERT/UPDATE;<br>业务过程 EXECUTE | 用于预订处理、入住办理、账单支付、清扫维修和绩效生成。 |
| <code>hotel_report</code>       | 报表查询用户 | 10 个视图 SELECT; 统计过程 EXECUTE                  | 只读取汇总视图和报表过程，不直接修改业务表。      |
| <code>hotel_customer_app</code> | 客户前台应用 | 客房/酒店/公告 SELECT; 预订、入住人、评价、收藏有限写入            | 满足客户浏览、预订、收藏和评价功能，限制后台表访问。  |

表 8 数据库用户与权限设计

```
CREATE USER IF NOT EXISTS 'hotel_admin'@'localhost' IDENTIFIED BY 'HotelAdmin@2026';
CREATE USER IF NOT EXISTS 'hotel_staff'@'localhost' IDENTIFIED BY 'HotelStaff@2026';
CREATE USER IF NOT EXISTS 'hotel_report'@'localhost' IDENTIFIED BY 'HotelReport@2026';
CREATE USER IF NOT EXISTS 'hotel_customer_app'@'localhost' IDENTIFIED BY
'HotelCustomer@2026';
GRANT ALL PRIVILEGES ON `hotel_management`.* TO 'hotel_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON `hotel_management`.`booking_order` TO
'hotel_staff'@'localhost';
GRANT SELECT, INSERT, UPDATE ON `hotel_management`.`checkin_record` TO
'hotel_staff'@'localhost';
GRANT SELECT, INSERT, UPDATE ON `hotel_management`.`bill` TO 'hotel_staff'@'localhost';
GRANT SELECT, INSERT, UPDATE ON `hotel_management`.`payment_record` TO
'hotel_staff'@'localhost';
GRANT EXECUTE ON PROCEDURE `hotel_management`.`sp_confirm_checkin` TO
'hotel_staff'@'localhost';
GRANT EXECUTE ON PROCEDURE `hotel_management`.`sp_pay_bill` TO 'hotel_staff'@'localhost';
```

```
GRANT SELECT ON `hotel_management`.`v_room_inventory_overview` TO
'hotel_report'@'localhost';
GRANT SELECT ON `hotel_management`.`v_bill_payment_summary` TO 'hotel_report'@'localhost';
GRANT EXECUTE ON PROCEDURE `hotel_management`.`sp_customer_consumption_report` TO
'hotel_report'@'localhost';

GRANT SELECT ON `hotel_management`.`room_catalog` TO 'hotel_customer_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON `hotel_management`.`booking_order` TO
'hotel_customer_app'@'localhost';
GRANT SELECT, INSERT ON `hotel_management`.`room_review` TO
'hotel_customer_app'@'localhost';
```

## 7.4 视图的创建

视图主要用于把多表关联结果封装为稳定查询对象，前端报表页面和数据库演示页面可以直接读取视图，减少页面层拼接 SQL 的复杂度。当前 SQL 共创建 10 个视图，覆盖房态库存、预订入住、账单支付、月度营收、员工操作、客户价值、清扫任务和员工绩效等场景。

| 视图名                          | 主要来源表                                                        | 作用说明                             |
|------------------------------|--------------------------------------------------------------|----------------------------------|
| v_room_inventory_overview    | room_catalog、room_instance                                   | 按客房目录统计总房间数、空闲数、预订数、在住数、清扫数和维修数。 |
| v_room_daily_status          | room_instance、booking_room、booking_order、checkin_record      | 展示实体房间当前状态及关联订单、入住、清扫、维修信息。      |
| v_booking_detail             | booking_order、customer、room_catalog、booking_room             | 汇总订单、客户、客房目录和实体房间分配情况。           |
| v_checkin_guest_detail       | checkin_record、checkin_guest、customer、employee、room_instance | 展示入住主记录、入住人和办理员工信息。              |
| v_bill_payment_summary       | bill、payment_record、refund_record                            | 汇总账单应付、已付、退款和状态。                 |
| v_monthly_revenue_report     | bill                                                         | 按月份统计应收实收、退款、净收入。                |
| v_employee_operation_summary | employee、operation_log                                       | 统计员工操作次数和最近操作时间。                 |
| v_customer_value_profile     | customer、booking_order、bill、review                           | 形成客户订单、消费和评价画像。                  |

| 视图名                           | 主要来源表                                        | 作用说明              |
|-------------------------------|----------------------------------------------|-------------------|
| v_housekeeping_task_detail    | housekeeping_task、<br>room_instance、employee | 展示清扫任务、房间和负责员工信息。 |
| v_employee_performance_detail | employee_performance、<br>employee            | 展示员工考核周期、指标和等级。   |

表 9 视图创建说明

```
CREATE VIEW `v_room_inventory_overview` AS
SELECT rc.id AS room_catalog_id, rc.jiudianmingcheng AS hotel_name,
       rc.kefangmingcheng AS room_name, rc.room_category, COUNT(ri.id) AS total_rooms,
       SUM(CASE WHEN ri.status='available' THEN 1 ELSE 0 END) AS available_rooms,
       SUM(CASE WHEN ri.status='reserved' THEN 1 ELSE 0 END) AS reserved_rooms,
       SUM(CASE WHEN ri.status='occupied' THEN 1 ELSE 0 END) AS occupied_rooms,
       ROUND(rc.kefangjiage,2) AS base_price, rc.review_count, rc.avg_score
FROM `room_catalog` rc LEFT JOIN `room_instance` ri ON ri.room_catalog_id=rc.id
GROUP BY rc.id, rc.jiudianmingcheng, rc.kefangmingcheng, rc.room_category, rc.kefangjiage,
rc.review_count, rc.avg_score;
```

```
CREATE VIEW `v_bill_payment_summary` AS
SELECT b.id AS bill_id, b.booking_id, b.checkin_id, b.customer_id, c.xingming AS
customer_name,
       b.room_amount, b.service_amount, b.discount_amount, b.payable_amount,
       COALESCE(SUM(CASE WHEN pr.pay_status='success' THEN pr.pay_amount ELSE 0 END),0) AS
actual_paid_amount,
       COALESCE(SUM(CASE WHEN rr.refund_status='success' THEN rr.refund_amount ELSE 0
END),0) AS actual_refund_amount,
       b.status AS bill_status, MAX(pr.paid_at) AS last_paid_at, b.created_at
FROM `bill` b LEFT JOIN `customer` c ON c.id=b.customer_id
LEFT JOIN `payment_record` pr ON pr.bill_id=b.id LEFT JOIN `refund_record` rr ON
rr.payment_id=pr.id
GROUP BY b.id, b.booking_id, b.checkin_id, b.customer_id, c.xingming, b.room_amount,
       b.service_amount, b.discount_amount, b.payable_amount, b.status, b.created_at;
```

```
CREATE VIEW `v_monthly_revenue_report` AS
SELECT DATE_FORMAT(`created_at`, '%Y-%m') AS `revenue_month`,
       SUM(`payable_amount`) AS `payable_total`,
       SUM(`paid_amount`) AS `paid_total`,
       SUM(`refund_amount`) AS `refund_total`,
       SUM(`paid_amount` - `refund_amount`) AS `net_revenue`
FROM `bill` GROUP BY DATE_FORMAT(`created_at`, '%Y-%m');
```

**客房库存总览视图**  
数据库: hotel\_management 视图: v\_room\_inventory\_overview 记录数: 7

| room_catalog_id | hotel_name      | room_name       | room_category | total_rooms | available_rooms | reserved_rooms | occupied_rooms | cleaning_rooms | maintenance_rooms | base_price | review_count | avg_score |
|-----------------|-----------------|-----------------|---------------|-------------|-----------------|----------------|----------------|----------------|-------------------|------------|--------------|-----------|
| 3001            | ShangriLa Hotel | RoomName1       | RoomType1     | 3           | 3               | 0              | 0              | 0              | 0                 | 122        | 0            | 0.00      |
| 3002            | ShangriLa Hotel | RoomName2       | RoomType2     | 2           | 0               | 2              | 0              | 0              | 0                 | 2          | 0            | 0.00      |
| 3003            | ShangriLa Hotel | RoomName3       | RoomType3     | 3           | 2               | 0              | 1              | 0              | 0                 | 3          | 0            | 0.00      |
| 3004            | ShangriLa Hotel | RoomName4       | RoomType4     | 4           | 1               | 3              | 0              | 0              | 0                 | 4          | 0            | 0.00      |
| 3005            | ShangriLa Hotel | RoomName5       | RoomType5     | 5           | 4               | 0              | 0              | 1              | 0                 | 5          | 0            | 0.00      |
| 3006            | ShangriLa Hotel | RoomName6       | RoomType6     | 6           | 4               | 1              | 0              | 1              | 0                 | 6          | 0            | 0.00      |
| 3007            | ShangriLa Hotel | river view room | DeluxeSuite   | 10          | 7               | 2              | 0              | 0              | 1                 | 666        | 0            | 0.00      |

当前截图由本地 hotel\_management 数据库的演示数据自动生成。

图 1 客房库存总览视图查询结果

**账单支付汇总视图**  
数据库: hotel\_management 视图: v\_bill\_payment\_summary 记录数: 1

| bill_id | booking_id | checkin_id | customer_id | customer_name | room_amount | service_amount | discount_amount | payable_amount | actual_paid_amount | actual_refund_amount | bill_status | last_paid_at        | created_at          |
|---------|------------|------------|-------------|---------------|-------------|----------------|-----------------|----------------|--------------------|----------------------|-------------|---------------------|---------------------|
| 9501    | 6008       | 9301       | 4001        | Customer1     | 122.00      | 20.00          | 5.00            | 137.00         | 137.00             | 0.00                 | paid        | 2026-05-08 10:00:00 | 2026-05-08 10:00:00 |

当前截图由本地 hotel\_management 数据库的演示数据自动生成。

图 2 账单支付汇总视图查询结果

## 7.5 存储过程的创建

## 7.5 存储过程的创建

存储过程把多表读写封装成可复用的数据库业务接口。后端 OperationalController 中的 /ops/dbops 接口会直接调用这些过程，前端 procedure\_center 页面则把过程封装为可操作的业务工具。当前脚本共包含 11 个过程，内容覆盖清扫、维修和员工绩效考核，已经超过早期分工文档中的范围。

| 存储过程名                       | 所属模块 | 输入参数概要              | 功能说明                    |
|-----------------------------|------|---------------------|-------------------------|
| sp_search_available_rooms   | 客房资源 | 房型、入住日期、离店日期、价格区间   | 检索符合日期和价格条件的可订实体房间。     |
| sp_batch_update_room_status | 客房资源 | 房型、原状态、新状态、操作人、备注   | 批量调整房态并写入房态日志和操作日志。     |
| sp_create_booking_order     | 预订入住 | 客户、客房目录、数量、预订日期、操作人 | 生成预订订单、分配可用实体房间锁定房态。    |
| sp_confirm_checkin          | 预订入住 | 订单、员工、入住人姓名、证件号、电话  | 办理入住，生成入住记录和入住人明细。      |
| sp_generate_checkout_bill   | 支付财务 | 入住记录、服务费、优惠金额、员工    | 根据入住信息生成退房账单。           |
| sp_pay_bill                 | 支付财务 | 账单、支付方式、支付金额、员工     | 登记支付流水，由触发器同步账单和订单支付状态。 |

| 存储过程名                           | 所属模块 | 输入参数概要            | 功能说明                      |
|---------------------------------|------|-------------------|---------------------------|
| sp_customer_consumption_report  | 统计报表 | 客户、起止日期           | 统计客户订单、消费和退款情况。           |
| sp_employee_performance_report  | 统计报表 | 员工、起止日期           | 统计员工入住、账单、支付、清扫、维修和操作次数。  |
| sp_create_housekeeping_task     | 后台服务 | 房间、员工、任务类型、备注     | 创建清扫任务并将房间置为 cleaning。    |
| sp_report_room_maintenance      | 后台服务 | 房间、报修员工、处理员工、问题描述 | 创建维修记录并将房间置为 maintenance。 |
| sp_generate_employee_assessment | 统计报表 | 员工、周期、考核人、管理评分    | 生成员工绩效记录，计算综合得分和等级。       |

表 10 存储过程创建说明

```

CREATE PROCEDURE `sp_create_booking_order`(
    IN p_customer_id bigint, IN p_room_catalog_id bigint, IN p_quantity int,
    IN p_booking_date date, IN p_operator_id bigint)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    SELECT COUNT(*) INTO v_available_count
    FROM `room_instance` ri
    WHERE ri.room_catalog_id=p_room_catalog_id AND ri.status='available'
    AND NOT EXISTS (
        SELECT 1 FROM `booking_room` br JOIN `booking_order` bo ON bo.id=br.booking_id
        WHERE br.room_id=ri.id AND br.room_status IN ('reserved','checked_in')
        AND bo.booking_status NOT IN ('cancelled','completed')
        AND br.stay_start_date < v_end_date AND br.stay_end_date > v_start_date
    );
    IF v_available_count < v_needed THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='not enough
available rooms'; END IF;
    INSERT INTO `booking_order`(..., ispay, booking_status, room_instance_id)
    VALUES(..., 'unpaid', 'created', v_room_id);
    INSERT INTO
    `booking_room`(booking_id,room_id,room_price,stay_start_date,stay_end_date,room_status)
    SELECT v_booking_id, room_id, v_price, v_start_date, v_end_date, 'reserved'
    FROM `tmp_bookings_selected_rooms`;
    INSERT INTO `booking_status_log`(booking_id,old_status,new_status,operator_id,remark)
    VALUES(v_booking_id,NULL,'created',p_operator_id,'booking created by procedure');

```

```
COMMIT;

END;

CREATE PROCEDURE `sp_pay_bill`(
    IN p_bill_id bigint, IN p_pay_method varchar(30), IN p_pay_amount decimal(10,2), IN
    p_employee_id bigint)
BEGIN
    IF p_pay_amount IS NULL OR p_pay_amount<=0 THEN SIGNAL SQLSTATE '45000' SET
    MESSAGE_TEXT='pay amount must be positive'; END IF;
    START TRANSACTION;
    SELECT payable_amount,paid_amount INTO v_payable,v_paid FROM `bill` WHERE id=p_bill_id
    FOR UPDATE;
    IF v_paid+p_pay_amount>v_payable THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='pay
    amount exceeds payable amount'; END IF;
    INSERT INTO
    `payment_record`(bill_id,payment_no,pay_method,pay_amount,pay_status,paid_at,remark)

    VALUES(p_bill_id,v_payment_no,IFNULL(p_pay_method,'cash'),p_pay_amount,'success',NOW(),
    'pay by procedure');
    INSERT INTO
    `operation_log`(operator_type,operator_id,business_type,business_id,action,remark)

    VALUES('employee',p_employee_id,'payment_record',LAST_INSERT_ID(),'pay_bill',v_payment_
    no);
    COMMIT;
END;

CALL sp_search_available_rooms('DeluxeSuite','2026-05-10','2026-05-12',300,1200);
CALL sp_create_booking_order(4007,3001,1,'2026-05-10',5001);
CALL sp_confirm_checkin(6007,5001,'HuYue','ID202605100001','13612512514');
CALL sp_generate_checkout_bill(9301,120.00,30.00,5001);
CALL sp_pay_bill(9501,'alipay',880.00,5001);
```

## 7.6 触发器的创建

触发器用于维护跨表联动。SQL 中共有 24 个触发器，其中支付触发器在更新账单状态的同时会同步 `booking_order.ispay`；退款触发器会同步 `booking_order.ispay`，当订单仍处于 `created` 且已全额退款时，还会将 `booking_status` 更新为 `cancelled`；客房评价和酒店评价触发器除更新平均评分外，也会写入 `operation_log`。

| 触发器名                                     | 触发时机             | 所属模块 | 主要作用                     |
|------------------------------------------|------------------|------|--------------------------|
| trg_booking_order_before_insert_validate | BEFORE<br>INSERT | 预订   | 校验客户、客房目录、实体房间和订单日期。     |
| trg_booking_order_before_update_validate | BEFORE<br>UPDATE | 预订   | 更新订单前再次校验关键外键和日期。        |
| trg_booking_guest_before_insert_validate | BEFORE<br>INSERT | 预订   | 校验预订入住人所属订单。             |
| trg_booking_guest_before_update_validate | BEFORE<br>UPDATE | 预订   | 更新预订入住人时保持订单有效。          |
| trg_checkin_guest_before_insert_validate | BEFORE<br>INSERT | 入住   | 校验入住人所属入住记录。             |
| trg_checkin_guest_before_update_validate | BEFORE<br>UPDATE | 入住   | 更新入住人时保持入住记录有效。          |
| trg_favorite_before_insert_validate      | BEFORE<br>INSERT | 收藏   | 校验收藏对象只能是客房或酒店。          |
| trg_favorite_before_update_validate      | BEFORE<br>UPDATE | 收藏   | 更新收藏时保持对象类型和外键有效。        |
| trg_payment_before_insert_validate       | BEFORE<br>INSERT | 支付   | 校验账单存在、支付金额和支付状态合法。      |
| trg_refund_before_insert_validate        | BEFORE<br>INSERT | 退款   | 校验原支付记录、退款金额和退款状态合法。     |
| trg_room_review_before_insert_validate   | BEFORE<br>INSERT | 评价   | 校验客房评价对象、客户和评分范围。        |
| trg_hotel_review_before_insert_validate  | BEFORE<br>INSERT | 评价   | 校验酒店评价对象、客户和评分范围。        |
| trg_booking_after_insert_lock_room       | AFTER INSERT     | 预订   | 原有订单创建后锁定实体房间并记录状态。      |
| trg_booking_after_update_release_room    | AFTER UPDATE     | 预订   | 订单取消或完成后释放房态。            |
| trg_booking_room_after_insert_lock_room  | AFTER INSERT     | 预订   | 订单房间明细插入后将房间置为 reserved。 |
| trg_checkin_after_insert_room_occupied   | AFTER INSERT     | 入住   | 入住记录生成后将房间置为 occupied。   |



| 触发器名                                       | 触发时机         | 所属模块 | 主要作用                                              |
|--------------------------------------------|--------------|------|---------------------------------------------------|
| trg_checkin_after_update_room_available    | AFTER UPDATE | 入住   | 退房后将房间置为 cleaning 并生成日志。                          |
| trg_housekeeping_after_update_release_room | AFTER UPDATE | 清扫   | 清扫完成后将房间恢复 available。                             |
| trg_maintenance_after_insert_mark_room     | AFTER INSERT | 维修   | 报修后将房间置为 maintenance。                             |
| trg_maintenance_after_update_release_room  | AFTER UPDATE | 维修   | 维修完成后将房间置为 cleaning。                              |
| trg_payment_after_insert_update_bill       | AFTER INSERT | 支付   | 成功支付后累计账单已付金额、更新账单状态，同步 booking_order.ispay 并写日志。 |
| trg_refund_after_insert_update_payment     | AFTER INSERT | 退款   | 成功退款后累计退款金额、更新账单状态，同步订单支付状态，必要时取消未入住订单。           |
| trg_room_review_after_insert_update_score  | AFTER INSERT | 评价   | 更新客房评论数和平均分，并写入 operation_log。                    |
| trg_hotel_review_after_insert_update_score | AFTER INSERT | 评价   | 更新酒店评论数和平均分，并写入 operation_log。                    |

表 11 触发器创建说明

```

CREATE TRIGGER `trg_payment_after_insert_update_bill`
AFTER INSERT ON `payment_record`
FOR EACH ROW
BEGIN
    IF NEW.pay_status='success' THEN
        SELECT payable_amount, paid_amount + NEW.pay_amount INTO v_payable, v_paid
        FROM `bill` WHERE id=NEW.bill_id;
        IF v_paid>=v_payable THEN SET v_new_status='paid';
        ELSEIF v_paid>0 THEN SET v_new_status='partial';
        ELSE SET v_new_status='unpaid'; END IF;
        UPDATE `bill` SET paid_amount=v_paid,status=v_new_status,updated_at=NOW() WHERE
id=NEW.bill_id;
        UPDATE `booking_order` bo JOIN `bill` b ON b.booking_id=bo.id
        SET bo.ispay=CASE WHEN v_new_status='paid' THEN 'paid'
                        WHEN v_new_status='partial' THEN 'partial' ELSE 'unpaid' END
        WHERE b.id=NEW.bill_id;
        INSERT INTO

```

```
`operation_log`(operator_type,operator_id,business_type,business_id,action,remark)
VALUES('system',NULL,'payment_record',NEW.id,'payment_inserted',NEW.payment_no);
END IF;
END;

CREATE TRIGGER `trg_refund_after_insert_update_payment`
AFTER INSERT ON `refund_record`
FOR EACH ROW
BEGIN
    IF NEW.refund_status='success' THEN
        SELECT bill_id INTO v_bill_id FROM `payment_record` WHERE id=NEW.payment_id;
        SELECT COALESCE(SUM(rr.refund_amount),0) INTO v_total_refund
        FROM `refund_record` rr JOIN `payment_record` pr ON pr.id=rr.payment_id
        WHERE pr.bill_id=v_bill_id AND rr.refund_status='success';
        SELECT paid_amount INTO v_paid_amount FROM `bill` WHERE id=v_bill_id;
        IF v_total_refund>=v_paid_amount THEN SET v_new_status='refunded'; ELSE SET
v_new_status='partial'; END IF;
        UPDATE `bill` SET refund_amount=v_total_refund,status=v_new_status,updated_at=NOW()
WHERE id=v_bill_id;
        UPDATE `booking_order` bo JOIN `bill` b ON b.booking_id=bo.id
        SET bo.ispay=CASE WHEN v_new_status='refunded' THEN 'refunded' ELSE 'partial' END,
        bo.booking_status=CASE WHEN v_new_status='refunded' AND
bo.booking_status='created'
THEN 'cancelled' ELSE bo.booking_status END
        WHERE b.id=v_bill_id;
    END IF;
END;

CREATE TRIGGER `trg_room_review_after_insert_update_score`
AFTER INSERT ON `room_review`
FOR EACH ROW
BEGIN
    SELECT COUNT(*), ROUND(AVG(score),2) INTO v_review_count, v_avg_score
    FROM `room_review` WHERE refid=NEW.refid;
    UPDATE `room_catalog` SET review_count=v_review_count,
avg_score=COALESCE(v_avg_score,0.00)
    WHERE id=NEW.refid;
    INSERT INTO
`operation_log`(operator_type,operator_id,business_type,business_id,action,remark)
VALUES('customer',NEW.userid,'room_review',NEW.id,'room_review_inserted',CONCAT('score=
',NEW.score));END;
```